CAIRO UNIVERSITY

Faculty of Graduate Studies for Statistical Research

Software Engineering Program — Professional Master's Degree

Predicting CI/CD Pipeline Build Failures Using Machine Learning Techniques

Presented by: **Moamen Mohamed Aly Hussein**

Student ID: 202401681

Supervisor: Dr. Maged Mamdouh

Date: Friday, September 11, 2026

Agenda

What we will cover in the next 20 minutes

1. **The Problem**
   Why CI/CD failures matter — financially and operationally
2. **Literature Gap**
   What prior research did, and what it left undone
3. **Proposed Solution**
   The Hybrid ML Pipeline architecture
4. **Methodology**
   Data, features, models, and evaluation regime
5. **Results**
   Headline metrics, ablation study, threshold optimization
6. **Business Impact**
   Translating model performance into dollar savings
7. **Conclusions & Future Work**
   What we proved, what remains

What Is CI/CD? And What Is GitHub?

The two ideas that everything else in this talk depends on

GITHUB

The shared archive

- **Where the world keeps its software:**
- Every change to the code is recorded
- Who changed what, and when
- A "commit" is one saved change
- Used by roughly 100 million developers
- Think: a very careful version-controlled filing cabinet

⚠ **WHY THIS COSTS MONEY**

Each check takes minutes to hours and costs real compute. When one fails, a developer must stop and fix it.

CI/CD

The automatic checking machine

- **Runs every time anyone saves a change:**
- Rebuilds the entire program from scratch
- Runs thousands of automated tests
- Reports back "passed" or "failed"
- No human involvement at any point
- Think: an automatic examiner that marks every draft

✓ **THE QUESTION THIS THESIS ASKS**

At the moment the change is saved — before the checking machine even starts — can we predict that it is going to fail?

Why This Problem Is Hard

One number shapes every methodological decision in this thesis

89%

of runs simply succeed

so guessing "it will pass" is already right 89% of the time

11%

actually fail

1,072 failures out of 9,772 collected runs

3.45

runs per commit

one saved change triggers many checks — this matters later

Why Accuracy Is The Wrong Measure Here

A model that predicts **"success" every single time** scores **89% accuracy** and has learned absolutely nothing. This thesis therefore reports **F1 on the failure class, which rewards only catching failures.**

The Problem

CI/CD failures are expensive — in time and money

11%

of all GitHub Actions runs

end in failure (observed in our dataset)

$0.008

per minute

GitHub Actions compute cost (Linux runner)

10–20 min

developer interruption

context-switch penalty per failure

The Cumulative Cost

For a mid-sized organization running **1,000 pipeline executions per day**, that is approximately **110 failed builds daily**, consuming compute resources and developer time that **cannot be recovered after the fact.**

Literature Gap

What prior research did — and what it missed

PRIOR WORK

Patel (2019) and related studies

- **Uses post-execution telemetry:**
- Run duration
- Resource utilization (CPU, memory)
- Step-level intermediate status
- Retry count and patterns
- Useful for retrospective anomaly detection

⚠ **CRITICAL LIMITATION**
By the time the predictor has its input, the resources have already been spent.

THIS PROJECT

Pre-execution prediction

- **Uses only pre-execution features:**
- Commit-level metadata
- Lines added / deleted / files changed
- Commit message (NLP via TF-IDF)
- Repository, branch, event context
- Enables proactive resource conservation

✓ **THE CONTRIBUTION**
Predict at commit time, before the runner starts. Decisions can prevent the cost rather than diagnose it.

Research Objectives

Four specific, measurable goals

01

Build a Hybrid ML Pipeline

Combine numerical, categorical, binary, and textual commit features into a single end-to-end classifier using scikit-learn's ColumnTransformer.

02

Compare Three Classifiers

Evaluate Logistic Regression (baseline), Random Forest (ensemble), and XGBoost (gradient boosting) under identical conditions.

03

Validate Under Realistic Conditions

Test the model on both stratified random and chronological splits to verify robustness to temporal data drift.

04

Quantify Business Impact

Translate precision and recall metrics into estimated annual cost savings under a documented operational scenario.

The Dataset

Real GitHub Actions data — not synthetic

DATASET PROFILE

9,772
Real workflow runs collected

18
Popular open-source repositories

89% / 11%
Success / Failure class balance

6
Programming languages covered

7,817 / 1,955
Train / Test split (stratified)

~2 hours
Collection time (rate-limited)

Sampled Repositories

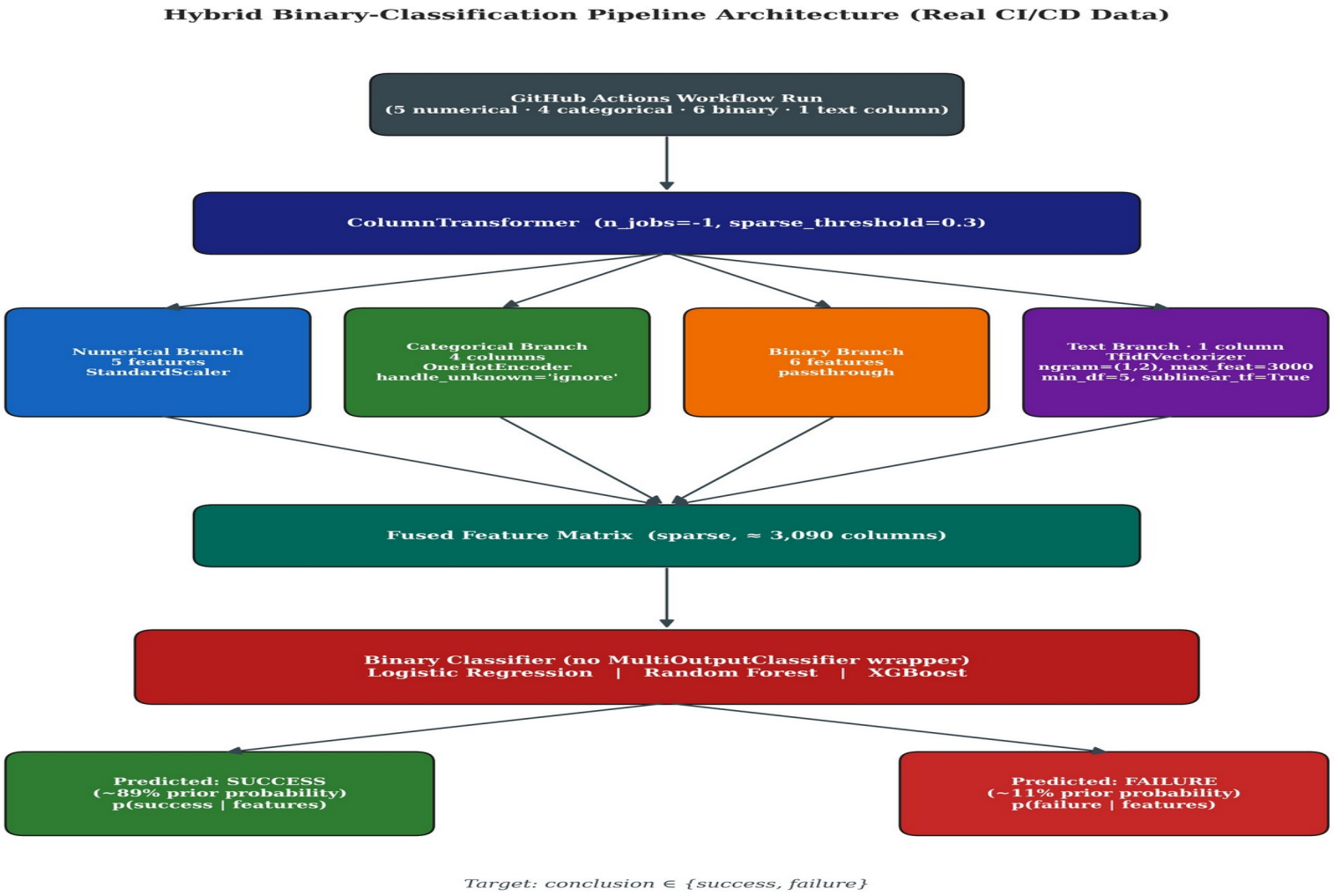
```
facebook/react
microsoft/vscode
tensorflow/tensorflow
pytorch/pytorch
rust-lang/rust
python/cpython
elastic/elasticsearch
prisma/prisma
...and 10 more
```

Why real data, not synthetic?

Initial experiments on a 45K-row synthetic Kaggle dataset revealed that columns were statistically independent — essentially random noise. Pivoting to real GitHub Actions data was the most important methodological decision in the project.

Hybrid Pipeline Architecture

Four parallel branches fused via ColumnTransformer



Key Design Choices

Parallel branches
Numerical (scaled), categorical (one-hot), binary (passthrough), text (TF-IDF) — processed independently then fused.

Sparse matrix
~3,090 columns total. Sparse-aware classifiers (LR, XGB) consume directly without densification.

Single classifier head
Binary output (no MultiOutputClassifier wrapper) — operational decision: success or failure.

Modular
Any branch can be swapped (e.g., TF-IDF → BERT) without touching the rest.

Feature Engineering

16 features across 4 modalities

NUMERICAL

5 features

- lines_added
- lines_deleted
- files_changed
- commit_age_days
- message_length

Treatment:

StandardScaler

CATEGORICAL

4 features

- repository
- workflow_name
- branch
- event

Treatment:

OneHotEncoder (with rare-category bucketing)

BINARY

6 features

- is_weekend
- is_night_commit
- is_pr_merge
- is_main_branch
- is_force_push
- is_dependabot

Treatment:

Passthrough (already 0/1)

TEXTUAL

1 feature

- cleaned commit message

Treatment:

TF-IDF (ngram 1-2, max_features=3000, sublinear_tf)

Identity-leakage defense: 693-token stoplist (author logins + repo names + bot signatures) prevents TF-IDF from learning identifiers.

Evaluation Regime

Commit-grouped cross-validation, threshold chosen off the test set

COMMIT-GROUPED 5-FOLD CV

Primary: what the thesis reports

- Every run of one commit stays on one side
- Threshold chosen on a validation fold, never on test
- Every row predicted once, by a model that never saw its commit
- *Answers: "How well does this generalise to unseen commits?"*

PER-REPOSITORY CHRONOLOGICAL

Secondary: deployment realism

- Each project cut at its own point in time
- Train on that project's past, test on its future
- Holds for all 18 of 18 projects; 57-day span
- *Answers: "Does it still work on later commits?"*

Metrics: Failure F1 (primary) · Balanced Accuracy · ROC-AUC · PR-AUC · Precision · Recall

Results — Default Threshold of 0.5

Three classifiers, commit-grouped cross-validation, threshold = 0.5

Model	Accuracy	Bal. Acc.	Precision	Recall	F1	ROC-AUC
Logistic Regression	76.3%	74.1%	27.6%	71.3%	0.397	0.828
Random Forest	83.0%	70.8%	33.4%	55.1%	0.416	0.801
XGBoost	90.4%	59.2%	73.1%	19.2%	0.304	0.824

📊 Key Observation

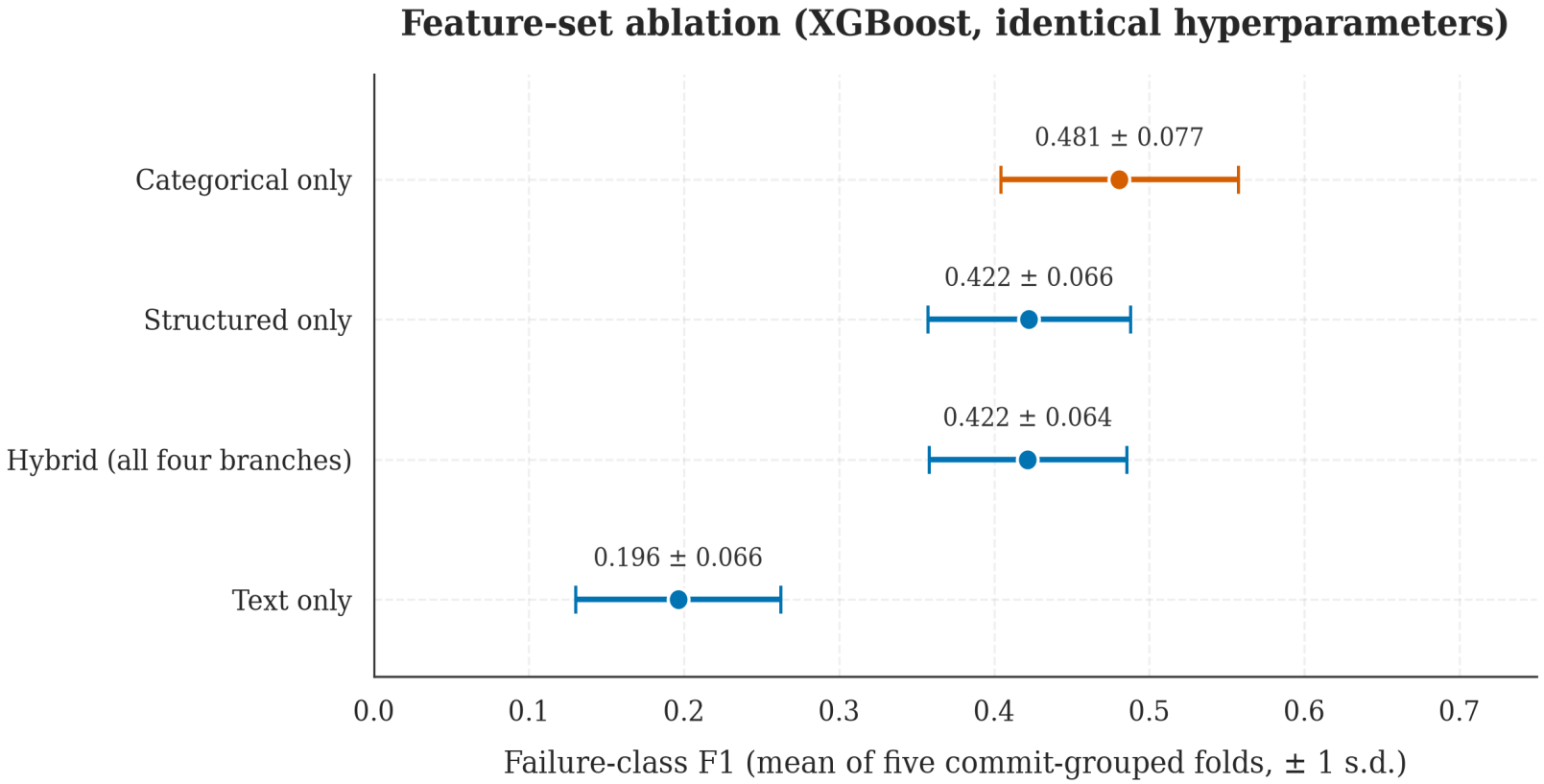
XGBoost achieves the highest PR-AUC (0.480) — meaning it ranks failures best — but the worst F1 (0.304).

Translation: the model has good information but a miscalibrated decision rule. Only 19% of true failures are flagged, because the 0.5 threshold is implicitly designed for a balanced class prior.

This insight motivates the threshold optimization experiment ahead →

Ablation Study

Does the hybrid claim hold? Test each feature set in isolation



Four Configurations

Text-only (TF-IDF)
F1 = 0.196 — cannot overcome the 89:11 prior

Structured-only
F1 = 0.422 — matches the full hybrid

Hybrid (full)
F1 = 0.422 — no lift from adding text

And note: the marginal ROC-AUC edge the hybrid held under the earlier protocol does not survive honest evaluation. Text does not improve ***ranking ability*** either. Means over five folds.

⚠ **Honest Finding**

The hybrid claim is refuted on real data. Adding TF-IDF text to the structured features produces no measurable lift.

The Discriminating Experiment

What if the model is told ONLY the project, workflow, branch and trigger?

Configuration	Failure F1	± s.d.	Precision	Recall	PR-AUC	ROC-AUC
Text only	0.196	0.066	0.313	0.156	0.208	0.665
Hybrid (full)	0.422	0.064	0.520	0.369	0.480	0.824
Categorical only	0.481	0.077	0.591	0.432	0.547	0.865

🔑 The Central Finding

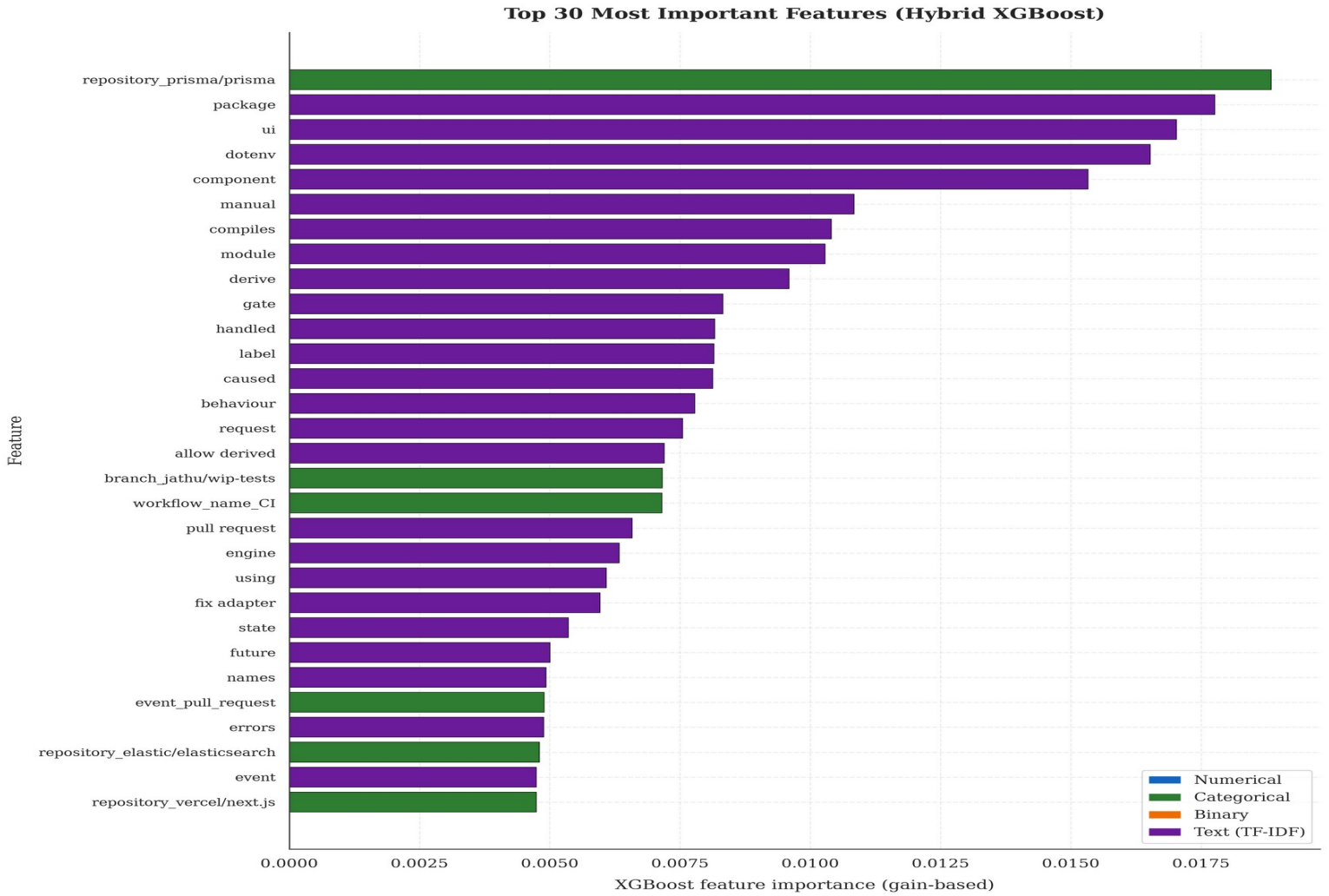
A model given no commit content at all — no size, no message — beats the full hybrid on every metric.

Honest reading: this system is largely a project-level risk estimator, not a commit-level one. Failure rates vary 38-fold across the eighteen projects, and Elasticsearch contributes 600 runs with zero failures.

I ran this experiment because it is the sharpest question an examiner can ask. →

Feature Importance

What does the model actually rely on?



Top Features

Repository identity ranks first

The single most important feature under the corrected split, up from rank 3 before.

TF-IDF tokens fill ranks 2–10

package, ui, dotenv — which part of the code was touched.

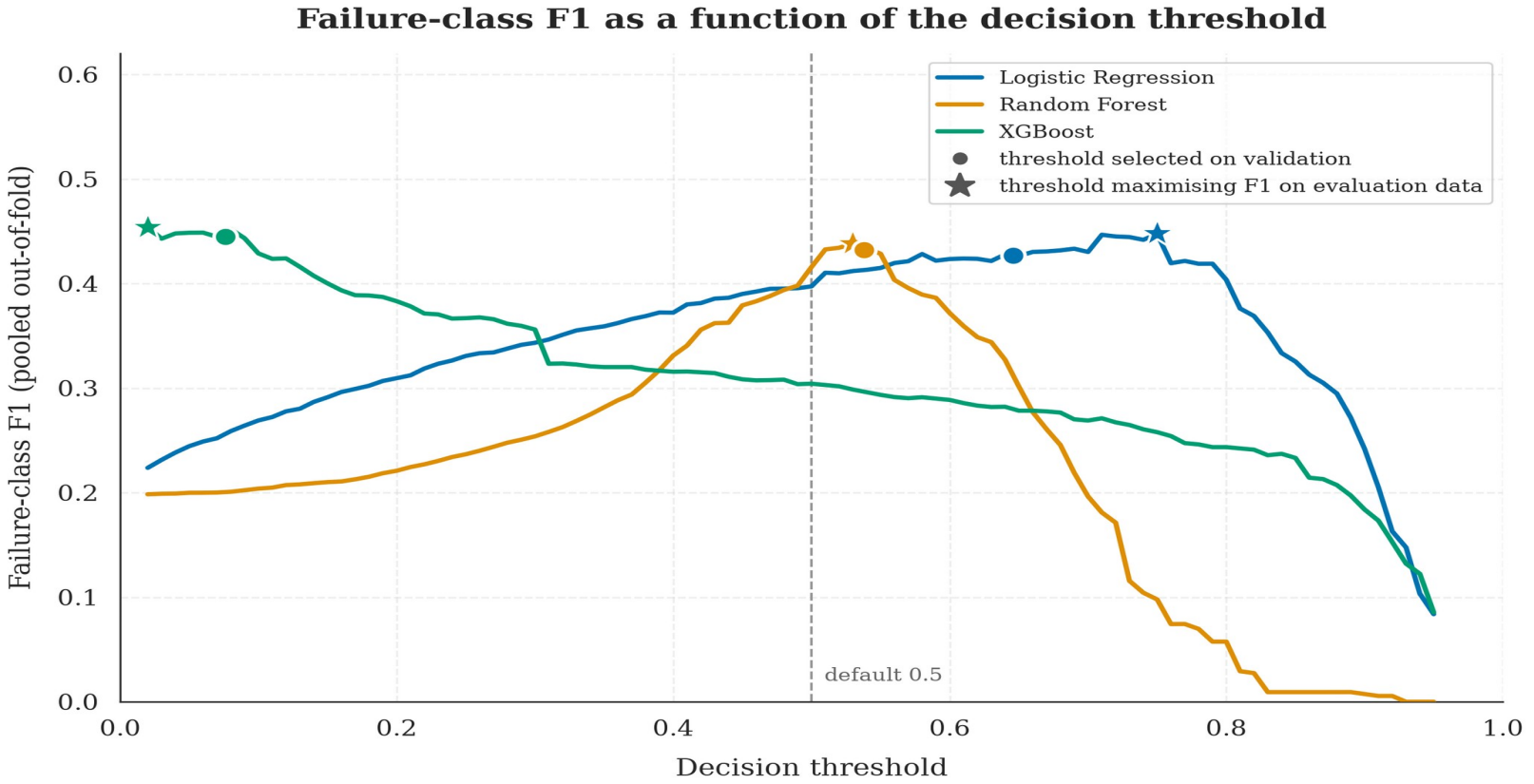
💡 *Consistent With The Ablation*

Failure-discriminative vocabulary is not "fix", "bug" or "revert" — but timezone, utc, thresholds, borrow.

Developers do not know at commit time that their build will fail. Yet the ablation shows text adds nothing once project identity is known.

Threshold Optimization

A real lever — but far smaller than first reported



Why this matters

XGBoost threshold selected at 0.076
An order of magnitude below the default 0.5.

Logistic Regression selected at 0.646
Above the default — opposite-direction miscalibration.

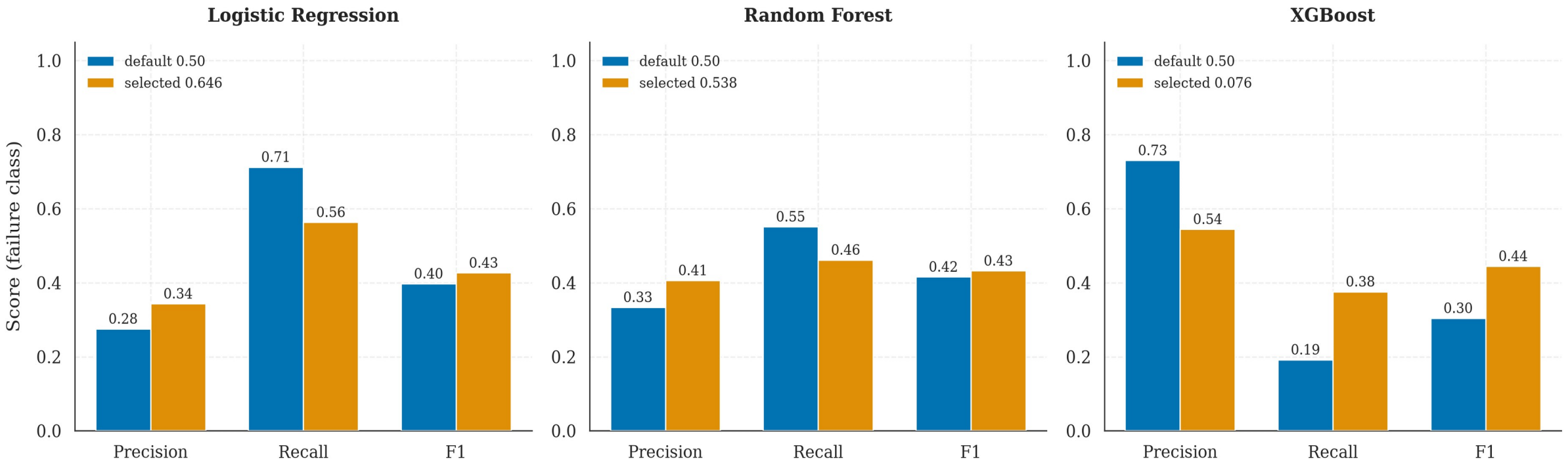
Random Forest selected at 0.538
Marginally above default.

+9.3 **percentage points**
honest improvement in failure F1

XGBoost F1 lifts from **0.304 → 0.397** · chosen on validation, **never on the test set**. *The +27pp figure came from picking it on the test set.*

Before vs. After Threshold Optimization

Per-model transformation



✓ Methodological lesson

For imbalanced binary classification, probability calibration is a first-class hyperparameter, not a deployment-time afterthought. But it must be calibrated on data that is not then used to report the result — and the next slide shows exactly what ignoring that was worth.

Where The Original Number Went

Same model, same data — only the measuring protocol changes

0.5924	−0.060	−0.126	0.4216
As reported in June	**Commit leakage**	**Threshold on test**	**The honest result**
row-level split, threshold picked on test	*same commit on both sides of the split*	*chosen and reported on the same data*	*grouped CV, threshold on validation*

The Result I Did Not Expect

Choosing the threshold on the test set cost **more than twice** *what the data leakage did.*

The leakage is the error everyone looks for; the threshold error is the one that actually cost more. I found it, measured it, corrected it, and published the correction.

FINAL RESULTS

XGBoost · threshold 0.076 · commit-grouped 5-fold cross-validation

0.422	66.9%	0.824	0.480
Failure F1	**Balanced Accuracy**	**ROC-AUC**	**PR-AUC**
± 0.064 across folds	*imbalance-aware*	*ranking ability*	*selection criterion*

Robustness Check (Per-Repository Chronological Split)

Failure F1 on the chronological test partition: **0.399** *(−2pp)*

Trained on each project's past and tested on its future, for all eighteen projects. The three classifiers are indistinguishable on F1; XGBoost is selected on PR-AUC.

Business Impact

From model metrics to dollar value

Operational Scenario

Mid-sized engineering organization

- **1,000** pipeline executions per day
- **11%** observed failure rate (~110/day)
- **$0.008/min** compute cost (GitHub published)
- **$75/hr** fully-loaded developer rate
- **15 min** context-switch penalty per failure
- **$2.50** false alarm triage cost
- **$18.75** missed failure cost

ESTIMATED ANNUAL NET SAVINGS

$243,670

per year, net of false alarms

Break-even false-alarm cost: $20.41

above which this configuration stops paying for itself. Logistic Regression looks better at $2.50 but breaks even at only $10.27.

Break-even > point estimate. It depends on a ratio I assumed, not measured.

Objectives — Achieved?

Validation against the four stated goals

01 **Build a Hybrid ML Pipeline** ✓ ACHIEVED

Four-branch ColumnTransformer implemented, ~3,090-column sparse matrix, <10ms inference latency on commodity hardware.

02 **Compare Three Classifiers** ✓ ACHIEVED

LR / RF / XGBoost trained on identical features, evaluated under identical metric suite, winner selected by objective criterion.

03 **Validate Under Realistic Conditions** ◐ PARTIALLY ACHIEVED

F1 = 0.422 grouped vs 0.399 chronological, temporally valid for all 18 projects. But the 600-run cap makes this short-horizon only.

04 **Quantify Business Impact** ✓ ACHIEVED

$243,670 annual net saving with documented assumptions, plus the break-even false-alarm cost at which it reaches zero.

Key Contributions

What this thesis adds to the field

Pre-execution prediction protocol

Strict feature-set discipline that distinguishes this work from post-execution telemetry approaches (Patel 2019).

Ablation identifying what actually drives the model

Categorical identity alone outperforms the full hybrid. The system is a project-level risk estimator, and this is reported rather than obscured.

A measured account of two evaluation defects

Commit leakage cost 0.060 of F1 and test-set threshold selection cost 0.126 — isolated under otherwise identical conditions.

Identity-leakage defense for TF-IDF

693-token stoplist algorithm that prevents the vectorizer from learning author/repo identifiers.

Full reproducibility

Fixed seed, pinned versions, every artefact committed. One command reproduces every reported number in about three minutes.

Business-impact translation

Precision and recall converted to dollars under explicit assumptions, reported with the break-even cost at which the saving vanishes.

Honest Limitations

What this work doesn't claim — and why that's important

Dataset scale

9,772 runs from 18 projects. Larger samples would tighten the confidence intervals; the spread across folds is currently ±0.06 of F1.

Open-source bias

Corporate CI/CD environments differ in commit cadence, branch policy, and tooling. Transfer is unverified.

TF-IDF text representation

Lexical-overlap encoding misses semantic similarity — though the ablation shows text contributes almost nothing regardless.

Preprocessing fitted on all data

Medians, vocabularies and the stoplist are computed before splitting. Target-independent, so mild, but they belong in the pipeline.

No production validation

Offline metrics ≠ deployed performance. A live deployment study would measure user behavior under predictions.

Temporal features computed in UTC

Computed in UTC across globally distributed projects — 18:00 UTC is mid-afternoon in California, midnight in Tokyo.

Future Work

Eight directions for extension

1. **Transformer-based text encoder**
   CodeBERT / Sentence-BERT — may restore hybrid advantage
2. **Scale to 1M+ workflow runs**
   Hundreds of repos, broader generalization
3. **Corporate dataset validation**
   Test transfer to proprietary CI/CD environments
4. **Online learning capability**
   Continuous adaptation to evolving codebases
5. **SHAP-based per-prediction explanations**
   Individual-commit interpretability
6. **Hyperparameter grid/Bayesian search**
   Tighter optimization than current defaults
7. **Live operationalized deployment**
   Measure real user behavior under predictions
8. **Multi-class failure stage prediction**
   Build vs. test vs. deploy failure differentiation

In Summary

01

Built a hybrid ML pipeline for pre-execution CI/CD failure prediction using only commit-time features available from the GitHub Actions API.

02

Measured F1 = 0.422 ± 0.064 under commit-grouped cross-validation with the threshold chosen off the test set — lower than the prior art, under stricter constraints and a stricter protocol.

03

Demonstrated through ablation that categorical identity alone outperforms the full hybrid, so the system is a project-level risk estimator rather than a commit-level one.

04

Quantified two evaluation defects in my own earlier results: 0.060 from commit leakage, 0.126 from threshold selection on test.

05

Estimated $243,670 annual net saving, reported alongside the break-even false-alarm cost of $20.41 at which it reaches zero.

Thank You

Questions & Discussion

Moamen Mohamed Aly Hussein

MSc Software Engineering · Cairo University

Defense: Friday, September 11, 2026

📎 **Backup material available:** *thesis document, source code, Q&A reference card*